DATA STRUCTURES AND ALGORITHMS

Simple Study Notes • PDF for testing a Retrieval-Augmented Generation (RAG) chatbot

1. Introduction to DSA

Data Structures and Algorithms (DSA) is a subject in computer science. A data structure is a way to store and organize data so that it can be used efficiently. An algorithm is a step-by-step method used to solve a problem. For example, a contact list is a way to organize names and phone numbers, while searching for a person's name is an operation performed on that data.

2. Array

An **array** is a linear data structure that stores elements of the same type in a sequence. Each element can be accessed using its index. In many programming languages, the first index is 0. Example: **[10, 20, 30, 40]**. The element at index 2 is 30. Arrays are useful when we need to store a fixed or indexed collection of values.

Common operations: Access an element, traverse the array, search for a value, insert an element, and delete an element.

3. Linked List

A **linked list** is a linear data structure made of nodes. Each node stores data and a link to the next node. Unlike an array, linked-list nodes do not need to be stored next to each other in memory. A singly linked list can be represented as: 10 → 20 → 30 → NULL. Linked lists are useful when insertions and deletions are frequent.

4. Stack

A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** rule. The last item added is the first item removed. Think of a stack of plates: the top plate is removed first. The main operations are **push** (add an item), **pop** (remove the top item), and **peek** (view the top item). Stacks are used in function calls, undo operations, and expression evaluation.

5. Queue

A **queue** is a linear data structure that follows the **First In, First Out (FIFO)** rule. The first item added is the first item removed. It works like a line of people waiting for a ticket. The main operations are **enqueue** (add an item at the rear) and **dequeue** (remove an item from the front). Queues are used in printer scheduling and task processing.

6. Searching Algorithms

Linear Search: Checks each element one by one until the required value is found or the list ends. It works on both sorted and unsorted data. Its worst-case time complexity is O(n).
Binary Search: Repeatedly divides the search range into two halves. It requires the data to be sorted. Its worst-case time complexity is O(log n).

7. Sorting Algorithms

Bubble Sort: Compares adjacent elements and swaps them if they are in the wrong order. Its average and worst-case time complexity is $O(n^2)$.
Selection Sort: Finds the smallest element in the unsorted part and places it in the correct position. Its time complexity is $O(n^2)$.
Merge Sort: Divides the array into smaller parts, sorts them, and merges the sorted parts. Its time complexity is O(n log n).

8. Time and Space Complexity

Time complexity describes how the running time of an algorithm grows as the input size increases. **Space complexity** describes how much extra memory an algorithm needs. Big-O notation is commonly used to express an upper bound on growth.

Notation	Name	Simple meaning
O(1)	Constant	Does not grow with input size.
O(log n)	Logarithmic	Grows slowly as input size increases.
O(n)	Linear	Grows in proportion to input size.
O(n log n)	Linearithmic	Common in efficient sorting algorithms.
$O(n^2)$	Quadratic	Often occurs with two nested loops.

9. Tree and Graph

A **tree** is a non-linear data structure made of nodes connected by edges. It has a root node, parent nodes, child nodes, and leaf nodes. A binary tree is a tree in which each node has at most two children. Tree traversal methods include inorder, preorder, and postorder.

A **graph** consists of vertices (also called nodes) and edges that connect them. Graphs can represent roads, computer networks, and social connections. **Breadth-First Search (BFS)** explores nearby vertices first and commonly uses a queue. **Depth-First Search (DFS)** explores one path deeply before backtracking and commonly uses a stack or recursion.

10. Quick Revision

Concept	Key point
Array	Elements are accessed using indexes.
Linked List	Nodes are connected using links.
Stack	LIFO: Last In, First Out.
Queue	FIFO: First In, First Out.
Linear Search	Checks elements one by one.
Binary Search	Needs sorted data; repeatedly halves the range.
BFS	Uses a queue.
DFS	Uses a stack or recursion.

End of notes. These notes are introductory and are intended for learning and testing a DSA question-answering chatbot.